

LLM Experiments on Kubeflow

Student Setup Guide

This guide gets your Kubeflow notebook ready for the LLM/NLP experiments in this course (LoRA, QLoRA, fine-tuning, RAG, embeddings, evaluation). The course environment is installed into a Python virtual environment (venv) that lives on your 15 GB NFS data volume, so it persists across pod restarts.

1. Before You Start — Create the Notebook Server

Log in to the Kubeflow dashboard at:

<https://kubeflow-csisrlab.bits-pilani.ac.in>

Click on “Login with BITS Pilani SSO”. Enter eLearn portal’s user credentials and Login.

Authentication

If you see HTTP 403 — Access Denied, open an Incognito window and log in again. Your session token may have expired. Clearing cookies also resolves this.

Check that your namespace is created and select it.

1.1 Create the 15 GB NFS Data Volume

This is your persistent working directory. All notebooks, venv, and scripts go here.

Navigate to: Kubeflow → Volumes → New Volume

Set these values exactly:

- Name: give it a recognisable name (e.g. llmall)
- Size: 15 GB
- Storage class: nfs-client
- Access mode: ReadWriteOnce

New Volume

Create a new empty Volume.

Name * Name Name cannot be empty

Namespace omshree-b-s2-25-aimlczg536

Volume size in Gi * 10

Storage Class * longhorn

Access Mode * ReadWriteOnce

CREATE CANCEL

⚠ PVC Quota — Maximum 5 Volumes

Your namespace is limited to 5 PersistentVolumeClaims total.

If you hit the limit (403 quota error), go to Volumes and delete unused workspace volumes.

Never delete your active 15 GB data volume — your work lives there.

1.2 Create the Notebook Server

Navigate to: Kubeflow → Notebooks → New Notebook

Configure as follows:

Field	Value
Name	any name (e.g. llmdemo)
Image	10.152.183.210:5000/bits-sudo-jupyter-pytorch-cuda-full
CPU / RAM	4/6, 6
GPU	1, NVIDIA
Workspace volume	6 GB (ephemeral — do not save work here)
Data volume	Attach the 15 GB NFS volume you just created
GPU toleration	Advanced options → Toleration group → L40S or A100

Workspace Volume

Volume that will be mounted in your home directory.

New volume

-workspace, Empty, 5Gi

Data Volumes

Additional volumes that will be mounted in your Notebook.

Existing volume

Type

Kubernetes Volume

☐ Readonly

Name *

l1mall	15Gi	ReadWriteOnce
l1mall-workspace	5Gi	ReadWriteOnce

/home/jovyan/

Configurations

Configurations

Allow access to Kubeflow Pipelines

Affinity / Tolerations

Affinity Config

None

Tolerations Group

None

NVIDIA A100 GPU Node

L40S-GPU

Miscellaneous Settings

☒ Enable Shared Memory



Advanced Options

Launch the Notebook.

Where Things Live

/home/jovyan/l1mall → Your 15 GB NFS data volume (PERSISTENT )

/home/jovyan/ → 6 GB workspace volume (WIPED on pod restart )

Rule: everything important goes inside ~/l1mall/

1.3 Common Startup Errors

Error	Fix
PodInitializing (2–5 min)	Normal — GPU nodes take time. Refresh every 30–60 sec.
nvidia-fabricmanager socket not found	GPU node is broken. Delete server, recreate with different GPU toleration, or contact admin.
PVC quota exceeded (403)	Delete unused volumes. Max 5 PVCs per namespace.
HTTP 403 on dashboard	Session expired. Open Incognito window and re-login.

2. Open Notebook and Navigate to Your Data Volume

STEP 1 Click Connect on your running notebook server

STEP 2 Open a Terminal: File → New → Terminal

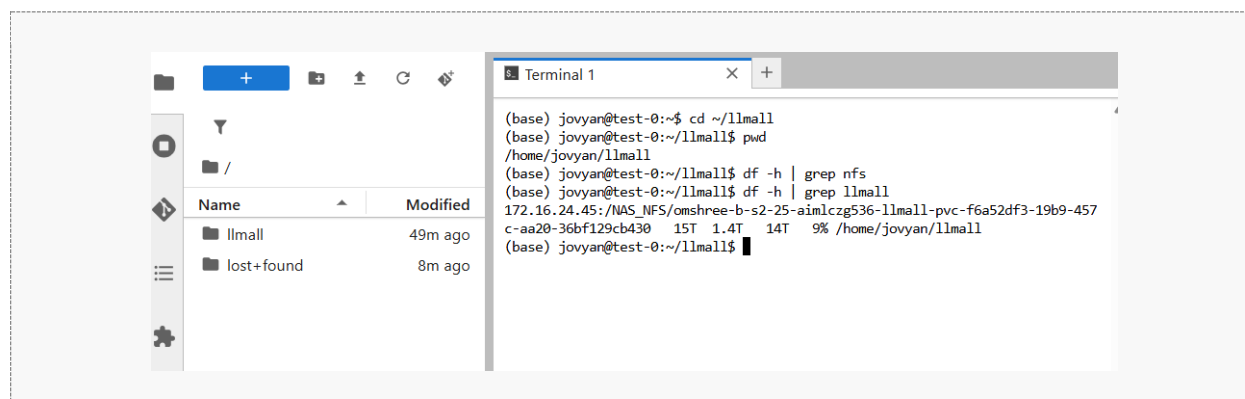
STEP 3 Navigate to your 15 GB NFS data volume

Your NFS data volume is mounted at ~/llmall (or whatever name you gave it). Confirm:

```
cd ~/llmall
pwd
# Expected: /home/jovyan/llmall
```

Verify it is the NFS volume (should show 15T available):

```
df -h | grep llmall
# Should show: 172.16.x.x:/NAS_NFS/... 15T ... /home/jovyan/llmall
```



⚠ Always cd into llmall first

Every terminal session starts in /home/jovyan/ (ephemeral).
Always run `cd ~/llmall` (data volume name) before any work.
The file browser left panel should show you are inside llmall/.

3. Upload and Run the Setup Scripts

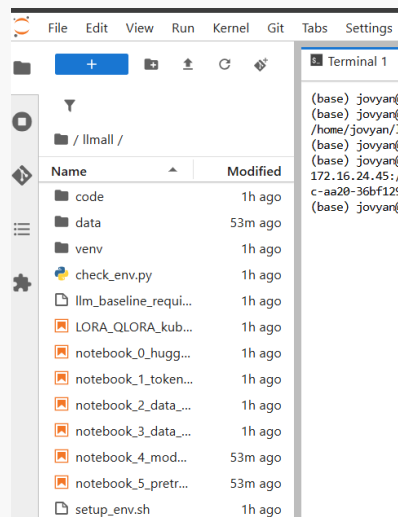
Two scripts drive the entire environment. Both must live inside `~/llmall/` (not inside the venv).

Script	Purpose
<code>setup_env.sh</code>	Creates the venv, installs all course packages, registers the Jupyter kernel
<code>check_env.py</code>	Verifies every package is installed at the correct version

3.1 Upload the Scripts

In the JupyterLab file browser (left panel):

1. Make sure you are inside the `llmall` folder (you should see `llmall/` in the breadcrumb at the top of the file browser)
2. Drag drop files or Click the Upload button (↑ arrow icon) in the file browser toolbar
3. Select `setup_env.sh` and `check_env.py`



⚠ Upload INTO llmall, not into /

Before clicking Upload, confirm the file browser breadcrumb shows `/ llmall /`

If it shows just `/` the files will land in the root filesystem and the script will not be found.

If this happens, move them: `mv /setup_env.sh ~/llmall/` and `mv /check_env.py ~/llmall/`

3.2 Run the Setup Script

In the terminal (make sure you are in `~/llmall`):

```
cd ~/llmall
bash ./setup_env.sh
```

This takes 2–4 minutes on first run. You will see 6 steps complete:

- Step 1/5 — Python version check
- Step 2/5 — Creating venv at ./venv
- Step 3/5 — Installing pinned course packages
- Step 4/5 — Register Jupyter kernel
- Step 5/5 — Saving versions to llm_baseline_requirements.txt

✓ Re-running after pod restart

The venv persists on your NFS volume — you do NOT need to re-run setup_env.sh after every restart.

Only re-run it if the Python (LLM venv) kernel is missing from the kernel picker.

It will detect the existing venv, skip reinstalling, and re-register the kernel in seconds.

3.3 Verify the Environment

```
./venv/bin/python ./check_env.py
```

Expected output ends with:

```
🐛 All checks passed – environment is ready!
```

⚠ If you see ⚠ warnings

Version mismatches are usually fine — the packages still work together.

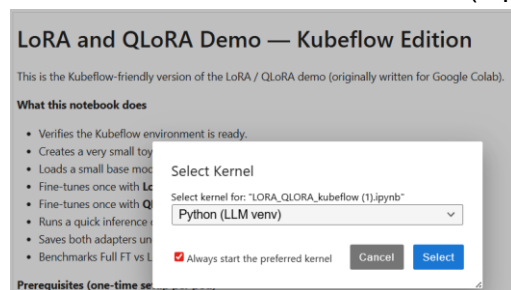
If you see ✗ MISSING, re-run `bash ./setup_env.sh` then run `check_env.py` again.

If torchao appears as installed, run: `./venv/bin/pip uninstall torchao -y`

4. Select the Kernel in Your Notebook

Installing the venv does NOT automatically change which Python your notebook uses. You must switch it manually.

- STEP 1 Open your notebook (.ipynb) from the llmall/ folder
- STEP 2 Click Kernel → Change Kernel in the menu bar
- STEP 3 Select Python (LLM venv) from the list
- STEP 4 The kernel indicator (top right) should show Python (LLM venv)



⚠ Most common mistake

If a notebook cannot import transformers, peft, or bitsandbytes, the wrong kernel is active. Always check the kernel indicator (top right of notebook) before running cells. The base kernel does NOT have the course packages — only Python (LLM venv) does.

5. File Paths in Notebooks

Kubeflow is NOT Colab. Paths like /content/... do not exist here.

Where	Correct path
Save model / adapter	<code>./data/my_adapter</code> or <code>os.path.abspath('./data/my_adapter')</code>
Training output dir	<code>./runs/run_1</code>
Dataset files	<code>./data/my_dataset.parquet</code>
Never use	<code>/content/...</code> or <code>~/...</code> or absolute paths starting with <code>/home/jovyan</code>

Use this pattern at the top of every notebook:

```
import os
WORK_DIR = os.path.abspath('./lora_experiments')
os.makedirs(WORK_DIR, exist_ok=True)

# Then in TrainingArguments:
#   output_dir = f'{WORK_DIR}/run_1'
# Saving adapters:
#   model.save_pretrained(f'{WORK_DIR}/lora_adapter')
```

✅ Why relative paths work here

As long as your notebook is opened from `~/llm/`, the path `./` resolves to `/home/jovyan/llm/` which is your persistent NFS volume. Your outputs survive pod restarts automatically.

6. Checking Available Memory

6.1 GPU Memory (most important for LLM training)

```
# Full GPU status — memory used, free, utilisation
nvidia-smi

# Live monitoring every 2 seconds (Ctrl+C to stop)
watch -n 2 nvidia-smi

# Compact one-liner
nvidia-smi --query-gpu=name,memory.used,memory.free,memory.total --
format=csv,noheader
```

6.2 GPU Memory from Inside a Notebook Cell

```
import torch

print('=== GPU Memory ===')
print(f'Allocated : {torch.cuda.memory_allocated()/1e9:.2f} GB')
print(f'Reserved  : {torch.cuda.memory_reserved()/1e9:.2f} GB')
total = torch.cuda.get_device_properties(0).total_memory / 1e9
print(f'Total      : {total:.2f} GB')
print(f'Free       : {total - torch.cuda.memory_reserved()/1e9:.2f} GB')
print(f'GPU       : {torch.cuda.get_device_name(0)}')
```

6.3 RAM and Disk

```
# RAM — total, used, free
free -h

# Disk — check how much of your 15 GB NFS volume is used
df -h | grep llmall

# See what is taking up space (run from ~/llmall)
du -sh * | sort -h
```

```
(base) jovyan@test-0:~/llmall$ free -h
              total        used        free      shared  buff/cache   availa
ble
Mem:           503Gi       18Gi       440Gi         67Mi        44Gi        48
1Gi
Swap:          8.0Gi         0B         8.0Gi
(base) jovyan@test-0:~/llmall$ df -h | grep lllmall
172.16.24.45:/NAS_NFS/omshree-b-s2-25-aimlczg536-llmall-pvc-f6a52df3-19b9-457
c-aa20-36bf129cb430 15T 1.4T 14T 9% /home/jovyan/llmall
(base) jovyan@test-0:~/llmall$ du -sh * | sort -h
8.0K  check_env.py
8.0K  setup_env.sh
12K   notebook_0_huggingface_auth.ipynb
20K   llm_baseline_requirements.txt
20K   notebook_3_data_packing.ipynb
24K   notebook_2_data_preparation.ipynb
24K   notebook_4_model_design.ipynb
52K   tokenizer_comparison.png
56K   LORA_QLORA_kubeflow (1).ipynb
57K   code
88K   notebook_5_pretraining_loop.ipynb
116K  notebook_1_tokenizer_comparison.ipynb
1.1G  venv
3.2G  data
```

6.4 Everything at Once — Quick One-Liner***

```
echo '=== GPU ===' && nvidia-smi --query-
gpu=name,memory.used,memory.free,memory.total --format=csv,noheader && echo '===
RAM ===' && free -h && echo '=== DISK ===' && df -h | grep lllmall
```

7. Save Old Notebooks and Free Up Space

Your 15 GB NFS volume fills up quickly with model weights (each model is 0.5–7 GB). Manage space proactively.

7.1 Check What Is Using Space

```
cd ~/llmall
du -sh * | sort -h
# Look for large folders — typically data/ and venv/
```

7.2 Download Finished Notebooks to Your Local Machine

4. In the JupyterLab file browser, right-click the finished .ipynb file
5. Click Download
6. Save it to your local machine
7. Verify it downloaded correctly
8. Only then delete it from the data volume

7.3 Delete Large Files to Free Space

Common things to delete when running low (in order of size saved):

What to delete	Command
Saved model checkpoints	<code>rm -rf ~/llmall/runs/</code>
Saved LoRA adapters (after downloading)	<code>rm -rf ~/llmall/data/lora_adapter/</code>
HuggingFace model cache	<code>rm -rf ~/.cache/huggingface/</code>
Old notebook outputs (clear outputs first)	Right-click → Clear All Outputs, then save
Downloaded base models in data/	<code>rm -rf ~/llmall/data/SmolLM2-26L-pruned-init/</code>

⚠ Never delete the venv unless you intend to reinstall

The venv is several GB but took 2–4 minutes to build.

If you delete it accidentally, just re-run `bash ./setup_env.sh` to rebuild.

Do NOT delete `llm_baseline_requirements.txt` — it records your exact package versions.

8. After a Pod Restart — Quick Recovery

Your installed packages survive restarts (they live on the NFS volume). You just need to re-select the kernel.

STEP 1 Open a terminal and cd into your data volume

```
cd ~/llmall
```

STEP 2 Open your notebook and re-select the kernel

Kernel → Change Kernel → Python (LLM venv)

STEP 3 If Python (LLM venv) is missing from the kernel list

```
cd ~/llmall
bash ./setup_env.sh    # re-registers kernel in ~10 seconds, skips reinstalling
```

Then: Ctrl+Shift+R (hard-refresh browser) → Kernel → Change Kernel → Python (LLM venv)

9. Troubleshooting

Symptom	Fix
Cannot import transformers / peft / bitsandbytes	Wrong kernel active. Kernel → Change Kernel → Python (LLM venv)
Python (LLM venv) not in kernel list	<code>cd ~/llmall && bash ./setup_env.sh</code> then hard-refresh browser
ImportError: cannot import name X from transformers	Base kernel is active, not venv. Check kernel indicator top-right.
CUDA Setup failed / bitsandbytes CUDA error	Re-run <code>bash ./setup_env.sh</code> . Confirm <code>nvidia-smi</code> shows GPU.
RuntimeError: No GPU found	Pod has no GPU. Recreate server with L40S or A100 toleration.
FileNotFoundError: /content/...	Colab path used. Replace with <code>os.path.abspath('./your_path')</code>
HFValidationError: Repo id must be 'namespace/name'	Use <code>os.path.abspath()</code> to convert relative model path to absolute.
Out of space on data volume	Run <code>du -sh * sort -h</code> to find large files. Delete old checkpoints/adapters.
nvidia-fabricmanager socket not found	GPU node broken. Recreate server with different GPU toleration or contact admin.
403 PVC quota exceeded	Delete unused volumes (max 5 PVCs). Kubeflow → Volumes → delete old workspace volumes.
HF_TOKEN not set in notebook	Run in terminal: <code>echo 'export HF_TOKEN=your_token' >> ~/llmall/.env</code> then in notebook: <code>import os; os.environ['HF_TOKEN']='your_token'</code>

10. Quick Reference Cheat Sheet

Situation	Command / Action
First-time setup	<code>cd ~/llmall && bash ./setup_env.sh</code>
Verify environment	<code>./venv/bin/python ./check_env.py</code>
Select kernel in notebook	Kernel → Change Kernel → Python (LLM venv)
After pod restart — select kernel	Open notebook → Kernel → Change Kernel → Python (LLM venv)
After pod restart — kernel missing	<code>cd ~/llmall && bash ./setup_env.sh</code> (re-registers in seconds)
Check GPU memory (terminal)	<code>nvidia-smi</code> or <code>watch -n 2 nvidia-smi</code>
Check GPU memory (notebook)	<code>torch.cuda.memory_allocated() / 1e9</code>
Check disk usage	<code>df -h grep llmall and du -sh * sort -h</code>
Free GPU memory after training	<code>del model; gc.collect(); torch.cuda.empty_cache()</code>
Download notebook before deleting	Right-click .ipynb → Download, then delete from volume
Clear notebook outputs to save space	Edit → Clear All Outputs → Save
Fix /content/... path error	Replace with <code>os.path.abspath('./your_folder')</code>
Fix relative path in from_pretrained	<code>model_path = os.path.abspath('./data/model_folder')</code>